

TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

Đề tài: Kiểm thử tự động Website Thương mại điện tử (E-commerce Web)

Nhóm thực hiện

Họ và tên	Mã sinh viên
-----------	--------------

Vũ Tuấn Anh	22010203
-------------	----------

Mai Đức Mạnh	23010814
--------------	----------

Môn học: Đánh giá và Kiểm định Chất lượng Phần mềm (LT3)

Giảng viên giảng dạy: Trịnh Thanh Bình

Hà Nội, tháng 6/2026

MỤC LỤC

ST T	Nội dung
1	Tóm tắt dự án
2	Phần I: Tài liệu đặc tả yêu cầu phần mềm (SRS)
2.1	Giới thiệu
2.2	Các tác nhân và phạm vi hệ thống

ST T	Nội dung
2.3	Yêu cầu chức năng
2.4	Luồng nghiệp vụ cốt lõi
2.5	Yêu cầu phi chức năng
3	Phần II: Kế hoạch kiểm thử phần mềm (Test Plan)
3.1	Mục tiêu kiểm thử
3.2	Phạm vi kiểm thử
3.3	Chiến lược kiểm thử
3.4	Tài nguyên và môi trường
3.5	Tiêu chí bắt đầu, kết thúc và rủi ro
4	Phần III: Thực hiện kiểm thử (Test Execution)
4.1	Môi trường thực thi
4.2	Kịch bản unit test
4.3	Kịch bản property-based test
4.4	Kịch bản integration test
4.5	Kịch bản E2E Selenium
4.6	Công cụ đo chất lượng
5	Phần IV: Báo cáo phân tích tổng hợp (Test Report)
5.1	Phạm vi thực thi
5.2	Thống kê kết quả
5.3	Báo cáo coverage
5.4	Defect log
5.5	Tổng kết và đề xuất cải tiến

TÓM TẮT DỰ ÁN

Trong bối cảnh các hệ thống thương mại điện tử ngày càng có nhiều chức năng tương tác trực tiếp với khách hàng, việc kiểm thử thủ công các luồng như đăng nhập, tìm kiếm sản phẩm, quản lý giỏ hàng, áp dụng mã giảm giá và đặt hàng dễ tốn thời gian, dễ bỏ sót lỗi hồi quy khi hệ thống thay đổi. Dự án này xây dựng một ứng dụng E-commerce Web bằng Spring Boot và đồng thời phát triển bộ kiểm thử tự động nhiều tầng để đánh giá chất lượng phần mềm một cách có hệ thống.

Ứng dụng sử dụng Spring Boot 3.2.5, Thymeleaf, Spring Security, Spring Data JPA và H2 in-memory database. Bộ kiểm thử sử dụng JUnit 5, Mockito, jqwik, MockMvc, Selenium WebDriver, JaCoCo, PIT Mutation Testing và GitHub Actions. Trọng tâm kiểm thử bao gồm service layer, controller layer và các luồng người dùng trên trình duyệt.

Mục tiêu của báo cáo là tổng hợp yêu cầu, kế hoạch kiểm thử, quá trình thực thi và kết quả kiểm thử hiện có của dự án. Báo cáo được hoàn thiện dựa trên mã nguồn, tài liệu trong thư mục docs/, cấu hình pom.xml, báo cáo test trong target/, báo cáo JaCoCo và ảnh chụp minh chứng từ Selenium.

Bảng phân chia trách nhiệm

ST T	Phân mục / Nhiệm vụ	Vũ Tuấn Anh	Mai Đức Mạnh	Trạng thái
1	Nghiên cứu nghiệp vụ E-commerce và xây dựng tài liệu SRS	x	x	Hoàn thành
2	Khảo sát hệ thống, lập kế hoạch kiểm thử	x	x	Hoàn thành
3	Xây dựng cấu trúc kiểm thử tự động bằng Maven/JUnit	x		Hoàn thành
4	Xây dựng unit test cho ProductService, CartService, OrderService	x	x	Hoàn thành
5	Xây dựng property-based test bằng jqwik		x	Hoàn thành
6	Xây dựng integration test bằng MockMvc	x	x	Hoàn thành

ST T	Phân mục / Nhiệm vụ	Vũ Tuấn Anh	Mai Đức Mạnh	Trạng thái
7	Xây dựng E2E test bằng Selenium WebDriver		x	Hoàn thành
8	Cấu hình JaCoCo, PIT và GitHub Actions	x	x	Hoàn thành
9	Tổng hợp test report và hoàn thiện báo cáo	x	x	Hoàn thành

Kế hoạch thực hiện

Giai đoạn	Nội dung	Kết quả
Phase 0	Thiết lập project, Spring Boot, Security, H2, Maven	Hoàn thành
Phase 1	Xây dựng tài liệu yêu cầu và kế hoạch kiểm thử	Hoàn thành
Phase 2	Hoàn thiện chức năng login, sản phẩm, giỏ hàng, checkout	Hoàn thành
Phase 3	Viết unit test, integration test, E2E test	Hoàn thành
Phase 4	Bổ sung BVA, EP, Decision Table, State Transition, PBT	Hoàn thành
Phase 5	Đo coverage bằng JaCoCo và cấu hình mutation testing bằng PIT	Hoàn thành
Phase 6	Tổng hợp demo guide, traceability matrix và báo cáo cuối	Hoàn thành

PHẦN I: TÀI LIỆU ĐẶC TẢ YÊU CẦU PHẦN MỀM (SRS)

1. Giới thiệu

1.1 Mục đích

Tài liệu đặc tả yêu cầu phần mềm mô tả các yêu cầu chức năng và phi chức năng của hệ thống E-commerce Web. Các yêu cầu này được dùng làm cơ sở để thiết kế test case, xây dựng ma trận truy xuất và đánh giá mức độ bao phủ kiểm thử.

Hệ thống phục vụ một quy trình mua sắm trực tuyến cơ bản: người dùng đăng nhập, xem danh sách sản phẩm, tìm kiếm/lọc sản phẩm, thêm sản phẩm vào giỏ hàng, áp dụng voucher, đặt hàng và theo dõi trạng thái đơn hàng.

1.2 Phạm vi

Phạm vi chức năng được xét trong báo cáo gồm:

- Quản lý người dùng: đăng nhập, đăng xuất, phân quyền, CSRF.
- Quản lý sản phẩm: hiển thị, tìm kiếm, lọc danh mục, thêm sản phẩm, cập nhật tồn kho, tính giá giảm.
- Quản lý giỏ hàng: thêm, xóa, xóa toàn bộ, tính tổng tiền, kiểm tra tồn kho.
- Đặt hàng và thanh toán: tạo đơn hàng, giảm giá theo số lượng, voucher hợp lệ/không hợp lệ/hết hạn, checkout với giỏ trống.
- Quản lý trạng thái đơn hàng: PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED.
- Giao diện: định dạng giá VND, nút hết hàng, thông báo khi không tìm thấy sản phẩm.

1.3 Từ điển thuật ngữ

Thuật ngữ	Giải thích
SRS	Software Requirement Specification, tài liệu đặc tả yêu cầu phần mềm
E2E	End-to-End, kiểm thử toàn bộ luồng nghiệp vụ qua giao diện trình duyệt

Thuật ngữ	Giải thích
Unit Test	Kiểm thử đơn vị, tập trung vào một hàm/lớp riêng lẻ
Integration Test	Kiểm thử tích hợp giữa controller và service/mocking dependencies
PBT	Property-Based Testing, kiểm thử dựa trên tính chất bất biến với dữ liệu sinh ngẫu nhiên
MockMvc	Công cụ kiểm thử Spring MVC mà không cần khởi động trình duyệt
Selenium	Công cụ tự động hóa trình duyệt cho E2E test
JaCoCo	Công cụ đo code coverage
PIT	Công cụ mutation testing
CSRF	Cross-Site Request Forgery, cơ chế bảo vệ form POST

2. Các tác nhân và phạm vi hệ thống

2.1 Tác nhân

Tác nhân	Vai trò
Khách vắng lai	Truy cập trang login nhưng chưa có quyền mua hàng
Người dùng đã đăng nhập	Xem sản phẩm, tìm kiếm, thêm giỏ hàng, checkout
Admin	Có quyền người dùng và quyền quản trị sản phẩm
Hệ thống kiểm thử tự động	Thực thi test case qua JUnit, MockMvc, jqwik và Selenium

2.2 Sơ đồ chức năng tổng quát

```

Hệ thống E-Commerce Web
|
+-- Quản lý người dùng
|   +-- Đăng nhập
|   +-- Đăng xuất
|   +-- Phân quyền
|   +-- Bảo vệ CSRF

```

```
|
+-- Quản lý sản phẩm
|   +-- Hiển thị danh sách sản phẩm
|   +-- Tìm kiếm theo tên
|   +-- Lọc theo danh mục
|   +-- Thêm sản phẩm
|   +-- Cập nhật tồn kho
|   +-- Tính giá giảm
|
+-- Giỏ hàng
|   +-- Thêm sản phẩm
|   +-- Xóa sản phẩm
|   +-- Xóa toàn bộ giỏ
|   +-- Tính tổng tiền
|   +-- Kiểm tra tồn kho
|
+-- Đặt hàng và thanh toán
|   +-- Tạo đơn hàng
|   +-- Giảm giá theo số lượng
|   +-- Áp dụng voucher
|   +-- Xử lý voucher sai/hết hạn
|   +-- Chặn checkout khi giỏ trống
|
+-- Trạng thái đơn hàng
    +-- PENDING -> CONFIRMED
    +-- CONFIRMED -> SHIPPED
    +-- SHIPPED -> DELIVERED
    +-- PENDING -> CANCELLED
```

3. Yêu cầu chức năng

3.1 Quản lý người dùng

ID	Yêu cầu	Mô tả
REQ-001	Đăng nhập	Người dùng nhập username/password để truy cập hệ thống
REQ-002	Đăng xuất	Người dùng đăng xuất, xóa session và chuyển về trang login
REQ-003	Phân quyền	Hệ thống phân biệt role USER và ADMIN

ID	Yêu cầu	Mô tả
REQ-004	Bảo mật CSRF	Mọi form POST phải có CSRF token hợp lệ

3.2 Quản lý sản phẩm

ID	Yêu cầu	Mô tả
REQ-005	Hiển thị danh sách sản phẩm	Trang chủ hiển thị sản phẩm từ database
REQ-006	Tìm kiếm sản phẩm	Tìm kiếm theo từ khóa trong tên sản phẩm, không phân biệt hoa/thường
REQ-007	Tìm kiếm theo danh mục	Lọc sản phẩm theo category, không phân biệt hoa/thường
REQ-008	Thêm sản phẩm	Admin thêm sản phẩm mới; tên không rỗng, giá > 0, stock >= 0
REQ-009	Cập nhật tồn kho	Tăng/giảm stock và không cho phép stock âm
REQ-010	Tính giá giảm	Tính giá sau discount trong khoảng 0% đến 100%, reject ngoài range

3.3 Giỏ hàng

ID	Yêu cầu	Mô tả
REQ-0011	Thêm vào giỏ	Thêm sản phẩm vào giỏ nếu đủ tồn kho, mỗi user có giỏ riêng
REQ-0012	Xóa khỏi giỏ	Xóa một sản phẩm khỏi giỏ, báo lỗi nếu sản phẩm không có trong giỏ
REQ-0013	Xóa toàn bộ giỏ	Xóa tất cả sản phẩm trong giỏ hàng
REQ-0014	Tính tổng giỏ	Tổng tiền bằng tổng giá nhân số lượng của tất cả item
REQ-001	Kiểm tra tồn	Không cho phép thêm vượt quá stock hiện có

ID	Yêu cầu	Mô tả
5	kho	

3.4 Đặt hàng và thanh toán

ID	Yêu cầu	Mô tả
REQ-01 6	Tạo đơn hàng	Tạo order từ cart items, giảm stock và tính tổng
REQ-01 7	Giảm giá theo số lượng	Dưới 10 item: 0%; từ 10 đến 19 item: 5%; từ 20 item trở lên: 10%
REQ-01 8	Áp dụng voucher	Mã hợp lệ làm giảm tổng tiền theo phần trăm tương ứng
REQ-01 9	Voucher không hợp lệ	Mã sai hiển thị lỗi rõ ràng
REQ-02 0	Voucher hết hạn	Mã hết hạn hiển thị lỗi hết hạn
REQ-02 1	Giỏ trống không checkout	Hiển thị cảnh báo khi checkout với giỏ trống

3.5 Quản lý trạng thái đơn hàng

ID	Yêu cầu	Mô tả
REQ-02 2	Trạng thái ban đầu	Đơn hàng mới tạo có status = PENDING
REQ-02 3	Xác nhận đơn	Chỉ cho phép PENDING -> CONFIRMED
REQ-02 4	Giao hàng	Chỉ cho phép CONFIRMED -> SHIPPED
REQ-02 5	Hoàn tất	Chỉ cho phép SHIPPED -> DELIVERED

ID	Yêu cầu	Mô tả
REQ-02 6	Hủy đơn	Chỉ cho phép PENDING -> CANCELLED và hoàn lại stock
REQ-02 7	Transition không hợp lệ	Chuyển trạng thái sai phải ném exception

3.6 Xem sản phẩm và giao diện

ID	Yêu cầu	Mô tả
REQ-02 8	Hiển thị giá format	Giá hiển thị dạng VND có dấu phân cách nghìn
REQ-02 9	Nút hết hàng	Sản phẩm stock = 0 hiển thị nút "Hết hàng" và disabled
REQ-03 0	Kết quả tìm kiếm trống	Không tìm thấy sản phẩm thì hiển thị thông báo phù hợp

4. Luồng nghiệp vụ cốt lõi

4.1 Luồng đăng nhập

Mục đích của luồng đăng nhập là xác thực người dùng trước khi sử dụng hệ thống mua sắm.

1. Người dùng truy cập trang login.
2. Hệ thống hiển thị form username và password.
3. Người dùng nhập thông tin hợp lệ.
4. Hệ thống xác thực bằng Spring Security.
5. Nếu thông tin đúng, người dùng được chuyển đến trang chủ.
6. Nếu thông tin sai, hệ thống giữ người dùng ở trang login và hiển thị lỗi.

Tài khoản test được seed/cấu hình trong hệ thống:

Username	Password	Role
standard_user	secret_sauce	USER

Username	Password	Role
admin_user	admin_pass	USER, ADMIN
performance_glitch_user	secret_sauce	USER

4.2 Luồng mua hàng và checkout

1. Người dùng đăng nhập bằng tài khoản hợp lệ.
2. Người dùng tìm kiếm sản phẩm, ví dụ từ khóa "Laptop".
3. Hệ thống hiển thị danh sách sản phẩm phù hợp.
4. Người dùng thêm một hoặc nhiều sản phẩm vào giỏ.
5. Hệ thống cập nhật số lượng ở badge giỏ hàng.
6. Người dùng mở giỏ hàng, kiểm tra sản phẩm và tổng tiền.
7. Người dùng nhập voucher, ví dụ SAVE10.
8. Hệ thống áp dụng giảm giá nếu voucher hợp lệ.
9. Người dùng đặt hàng.
10. Hệ thống tạo đơn hàng, giảm tồn kho, xóa giỏ hàng và hiển thị trang xác nhận.

4.3 Luồng trạng thái đơn hàng

```
PENDING
|-- confirmOrder() --> CONFIRMED
|-- cancelOrder() --> CANCELLED

CONFIRMED
|-- shipOrder() --> SHIPPED

SHIPPED
|-- deliverOrder() --> DELIVERED
```

Các chuyển trạng thái ngoài sơ đồ trên bị xem là không hợp lệ và phải ném `InvalidOrderStateException`.

5. Yêu cầu phi chức năng

Nhóm yêu cầu	Mô tả
Bảo mật	Sử dụng Spring Security, session authentication và CSRF token

Nhóm yêu cầu	Mô tả
	cho form POST
Tính đúng đắn	Tính tổng tiền, discount, voucher và stock phải chính xác
Tính ổn định	Unit/integration/E2E test phải chạy ổn định trên môi trường Maven
Khả năng kiểm thử	Các trang Selenium dùng Page Object Model để giảm trùng lặp
Khả năng tự động hóa	GitHub Actions chạy test và upload báo cáo JaCoCo/Surefire/Failsafe
Khả năng quan sát	Kết quả test, screenshots và coverage report được lưu trong target/

PHẦN II: KẾ HOẠCH KIỂM THỬ PHẦN MỀM (TEST PLAN)

1. Mục tiêu kiểm thử

Mục tiêu kiểm thử là xác nhận các chức năng chính của hệ thống E-commerce hoạt động đúng theo yêu cầu, phát hiện lỗi logic ở service layer, lỗi xử lý HTTP ở controller layer và lỗi hành vi người dùng ở giao diện trình duyệt.

Cụ thể, kế hoạch kiểm thử hướng đến:

- Đảm bảo login, logout, phân quyền và CSRF hoạt động đúng.
- Đảm bảo sản phẩm hiển thị/tìm kiếm/lọc đúng dữ liệu.
- Đảm bảo giỏ hàng không cho thêm sản phẩm vượt tồn kho và tổng tiền được tính chính xác.
- Đảm bảo voucher hợp lệ, voucher không hợp lệ và voucher hết hạn được xử lý đúng.
- Đảm bảo đơn hàng tuân thủ state transition đã đặc tả.
- Đảm bảo luồng E2E mua hàng hoàn chỉnh chạy được qua trình duyệt.
- Đảm bảo service layer đạt ngưỡng coverage line $\geq 80\%$ và branch $\geq 70\%$.

2. Phạm vi kiểm thử

2.1 Trong phạm vi

Nhóm kiểm thử	Phạm vi
Unit Test	ProductService, CartService, OrderService
Property-Based Test	Bất biến của tổng đơn hàng và giá sau giảm
Integration Test	HomeController, CartController, CheckoutController bằng MockMvc
E2E Test	Tìm kiếm, giỏ hàng, voucher, checkout bằng Selenium
White-box Coverage	Đo line/branch coverage bằng JaCoCo
Mutation Testing	Cấu hình PIT cho service layer
CI/CD	GitHub Actions chạy test và upload artifacts

2.2 Ngoài phạm vi

Hạng mục	Lý do
Performance/Load Test	Chưa có yêu cầu đo tải đồng thời
Security Penetration Test	Dự án tập trung vào kiểm thử chức năng và regression
Usability Test	Chưa có tiêu chí đánh giá UX định lượng
Thanh toán thật qua ví/thẻ	Hệ thống demo không tích hợp cổng thanh toán thật

3. Chiến lược kiểm thử

3.1 Testing pyramid

/\	
/E2E\	8 cases - Selenium WebDriver
/----\	
/Integ \	15 cases - MockMvc

```

/-----\
/Unit+PBT \      61 cases - JUnit 5 + Mockito + jqwik
/-----\

```

Tổng hiện tại: 84 automated tests

3.2 Mức độ kiểm thử

Mức kiểm thử	Công cụ	Mục tiêu	Số lượng hiện tại
Unit Test	JUnit 5 + Mockito	Kiểm thử logic service layer	55
Property-Based Test	jqwik 1.8.4	Kiểm thử bất biến với dữ liệu sinh ngẫu nhiên	6
Integration Test	MockMvc	Kiểm thử controller request/response	15
E2E Test	Selenium WebDriver	Kiểm thử luồng người dùng qua browser	8
Tổng	Maven Surefire/Failsafe	Regression suite tự động	84

3.3 Kỹ thuật thiết kế test case

Kỹ thuật	Áp dụng cho	Ví dụ trong dự án
Equivalence Partitioning (EP)	Dữ liệu hợp lệ/không hợp lệ	Tên sản phẩm rỗng, giá âm, voucher sai
Boundary Value Analysis (BVA)	Giá trị biên	price = 0, stock = 0, qty = 0, discount = 0%/100%, số lượng 9/10/19/20
Decision Table	Tổ hợp điều kiện voucher	SAVE10/SAVE20/SAVE50, INVALID, EXPIRED2024
State Transition	Vòng đời đơn hàng	PENDING -> CONFIRMED -> SHIPPED -> DELIVERED
Property-Based Testing	Bất biến tính toán	Total không âm, total không vượt subtotal, discount cao làm giá thấp hơn

Kỹ thuật	Áp dụng cho	Ví dụ trong dự án
E2E Scenario	Luồng nghiệp vụ người dùng	Login -> Add to cart -> Apply voucher -> Place order

4. Tài nguyên và môi trường

4.1 Công nghệ hệ thống

Thành phần	Công nghệ	Phiên bản / Ghi chú
Framework	Spring Boot	3.2.5
Template engine	Thymeleaf	Spring Boot starter
Authentication	Spring Security	Session + CSRF
Database	H2 in-memory	Tự reset khi test
ORM	Spring Data JPA	Repository layer
Build tool	Maven	3.9+
Java	JDK	17+

4.2 Công cụ kiểm thử

Công cụ	Mục đích
JUnit 5	Framework chạy unit/integration tests
Mockito	Mock repository/service dependencies
jqwik	Property-based testing
MockMvc	Kiểm thử controller mà không cần browser
Selenium WebDriver	Tự động hóa trình duyệt
WebDriverManager	Quản lý ChromeDriver
JaCoCo	Đo line/branch coverage

Công cụ	Mục đích
PIT	Mutation testing cho service layer
GitHub Actions	CI/CD test automation

4.3 Dữ liệu test

Nhóm dữ liệu	Nội dung
User	standard_user, admin_user, performance_glitch_user
Voucher hợp lệ	SAVE10, SAVE20, SAVE50
Voucher hết hạn	EXPIRED2024
Sản phẩm seed	6 sản phẩm thuộc nhóm electronics/fashion
Database test	H2 in-memory, profile test

5. Tiêu chí bắt đầu, kết thúc và rủi ro

5.1 Entry criteria

Tiêu chí	Trạng thái
Project Maven biên dịch được	Đạt
Cấu hình H2 test database sẵn sàng	Đạt
Spring Security login và CSRF đã cấu hình	Đạt
Test data được seed	Đạt
Chrome/ChromeDriver sẵn sàng cho Selenium	Đạt trên môi trường đã chạy report

5.2 Exit criteria

Tiêu chí	Ngưỡng	Trạng thái hiện tại
Tất cả automated tests pass	0 failures/errors	Đạt
Service line coverage	$\geq 80\%$	Đạt, khoảng 97.4%
Service branch coverage	$\geq 70\%$	Đạt, khoảng 91.2%
Requirement coverage	30/30 yêu cầu có test hoặc cấu hình cover	Đạt
CI workflow	Có workflow chạy Maven test/verify	Đạt

5.3 Rủi ro và biện pháp giảm thiểu

Rủi ro	Khả năng	Tác động	Biện pháp
Selenium flaky do timing	Trung bình	E2E fail không ổn định	Dùng WebDriverWait và locator ổn định
CSRF gây lỗi POST trong test	Cao	Controller/E2E fail 403	Dùng csrf() trong MockMvc và form Thymeleaf đúng th:action
PIT chạy lâu	Trung bình	Tốn thời gian demo	Giới hạn targetClasses=com.ecommerce.service.*, dùng threads=4
ChromeDriver không tương thích	Trung bình	Selenium không khởi động	Dùng WebDriverManager và Chrome stable
Dữ liệu test bị phụ thuộc thứ tự	Thấp	Test khó bảo trì	Seed dữ liệu rõ ràng, reset H2 khi test

PHẦN III: THỰC HIỆN KIỂM THỬ (TEST EXECUTION)

1. Môi trường thực thi

Thành phần	Giá trị
Hệ điều hành	Linux/Windows đều có thể chạy bằng Maven
Ngôn ngữ	Java 17+
Build	Maven
Backend	Spring Boot 3.2.5
Database	H2 in-memory
Browser E2E	Chrome headless
Test runner	Maven Surefire và Maven Failsafe
Report	Surefire reports, Failsafe reports, JaCoCo HTML/CSV/XML, screenshots

Các lệnh thực thi chính:

```
mvn compile
mvn test -B
mvn verify -B
mvn test jacoco:report -B
mvn org.pitest:pitest-maven:mutationCoverage -B
```

2. Kịch bản unit test

2.1 ProductServiceTest

Nhóm kiểm thử	Mục tiêu	Kỹ thuật
Thêm sản phẩm hợp lệ	Kiểm tra sản phẩm được lưu khi name/price/stock/category hợp lệ	EP
Tên rỗng/null	Ném InvalidInputException	EP
Giá âm hoặc bằng 0	Chặn input sai	BVA
Stock âm	Chặn tồn kho không hợp lệ	BVA
Cập nhật stock	Stock tăng/giảm đúng và không âm	EP/BVA
Tìm kiếm danh mục	Không phân biệt hoa/thường	EP
Tính giá discount	Discount trong khoảng 0-100 hợp lệ, ngoài khoảng bị reject	BVA
Số test hiện tại: 15. Kết quả trong target/surefire-reports/com.ecommerce.unit.ProductServiceTest.txt: 15 pass, 0 failures.		

2.2 CartServiceTest

Nhóm kiểm thử	Mục tiêu	Kỹ thuật
Thêm vào giỏ khi đủ stock	Cart có item đúng product và quantity	EP
Stock = 0	Không cho thêm sản phẩm hết hàng	BVA
Quantity vượt stock	Ném InsufficientStockException	BVA
Quantity = 0 hoặc âm	Ném InvalidInputException	BVA
Xóa item khỏi giỏ	Item biến mất khỏi cart	EP
Xóa item không tồn tại	Ném lỗi phù hợp	EP
Xóa toàn bộ giỏ	Cart trống	EP

Nhóm kiểm thử	Mục tiêu	Kỹ thuật
Tính tổng cart	Tổng bằng tổng item price nhân quantity	EP
Thêm cùng sản phẩm hai lần	Quantity được cộng dồn	EP
Số test hiện tại: 13.	Kết quả trong	target/surefire-reports/com.ecommerce.unit.CartServiceTest.txt: 13 pass, 0 failures.

2.3 OrderServiceTest

Nhóm kiểm thử	Mục tiêu	Kỹ thuật
Tạo order đủ stock	Order tạo thành công, stock giảm	EP
Tạo order thiếu stock	Ném InsufficientStockException	BVA
Cart null/rỗng	Ném InvalidInputException	EP
Giảm giá theo số lượng	Dưới 10: 0%, 10-19: 5%, từ 20: 10%	EP/BVA
Biên số lượng	9, 10, 19, 20 item	BVA
Voucher hợp lệ	SAVE10/SAVE20/SAVE50 giảm đúng tổng tiền	Decision Table
Voucher sai/hết hạn	INVALID/EXPIRED2024 ném lỗi	Decision Table
Hủy đơn	PENDING -> CANCELLED và hoàn stock	State Transition
Xác nhận/giao/hoàn tất	PENDING -> CONFIRMED -> SHIPPED -> DELIVERED	State Transition
Transition sai	Ném InvalidOrderStateException	State Transition
Số test hiện tại: 27.	Kết quả trong	target/surefire-reports/com.ecommerce.unit.OrderServiceTest.txt: 27 pass, 0 failures.

3. Kịch bản property-based test

Property-based tests dùng jqwik để sinh nhiều dữ liệu ngẫu nhiên và kiểm tra tính chất bất biến của nghiệp vụ. Đây là phần bổ sung cho unit test truyền thống, giúp phát hiện lỗi trên nhiều tổ hợp input hơn.

TC	Tính chất kiểm tra	Số lần sinh dữ liệu	Kỳ vọng
PBT-001	Order total luôn không âm	200	<code>total >= 0</code>
PBT-002	Order total không vượt subtotal	200	<code>total <= subtotal</code>
PBT-003	Thêm item làm subtotal không giảm	100	Monotonic subtotal
PBT-004	Tỷ lệ discount chỉ thuộc 1.0, 0.95, 0.90	200	Không có rate ngoài chính sách
PBT-005	Giá sau discount nằm trong khoảng hợp lệ	200	<code>0 <= result <= price</code>
PBT-006	Discount cao hơn làm giá thấp hơn	100	Monotonic discount

Số test hiện tại: 6. Kết quả trong target/surefire-reports/com.ecommerce.unit.PropertyBasedTest.txt: 6 pass, 0 failures.

4. Kịch bản integration test

Integration tests dùng MockMvc để kiểm thử request/response của controller layer mà không cần khởi động trình duyệt thật.

4.1 HomeControllerTest

Kịch bản	Expected
Người dùng authenticated vào trang chủ	HTTP 200, model có danh sách sản phẩm
Tìm kiếm với keyword	Model trả về danh sách đã lọc

Kịch bản	Expected
Tìm kiếm keyword rỗng	Trả về toàn bộ sản phẩm
Add to cart bằng POST	Redirect về trang chủ
Số test hiện tại: 4. Kết quả: 4 pass.	

4.2 CartControllerTest

Kịch bản	Expected
Xem giỏ có item	HTTP 200, model có cart, cartEmpty=false
Xem giỏ trống	HTTP 200, cartEmpty=true
Xóa sản phẩm khỏi giỏ	Redirect về /cart
Clear cart	Redirect về /cart
Số test hiện tại: 4. Kết quả: 4 pass.	

4.3 CheckoutControllerTest

Kịch bản	Expected
Checkout với giỏ trống	Hiển thị warning và orderTotal=0
Checkout với giỏ có item	Hiển thị tổng tiền
Apply voucher hợp lệ	Redirect và flash success
Apply voucher không hợp lệ	Redirect và flash error
Place order thành công	Redirect đến trang confirmation, cart được clear
Place order với giỏ trống	Redirect về checkout và warning
Order confirmation	HTTP 200, model có order
Số test hiện tại: 7. Kết quả: 7 pass.	

5. Kịch bản E2E Selenium

Selenium tests sử dụng `@SpringBootTest(webEnvironment = RANDOM_PORT)` để khởi động ứng

dùng trên port ngẫu nhiên, sau đó điều khiển Chrome headless qua Page Object Model.

5.1 Cấu trúc Page Object Model

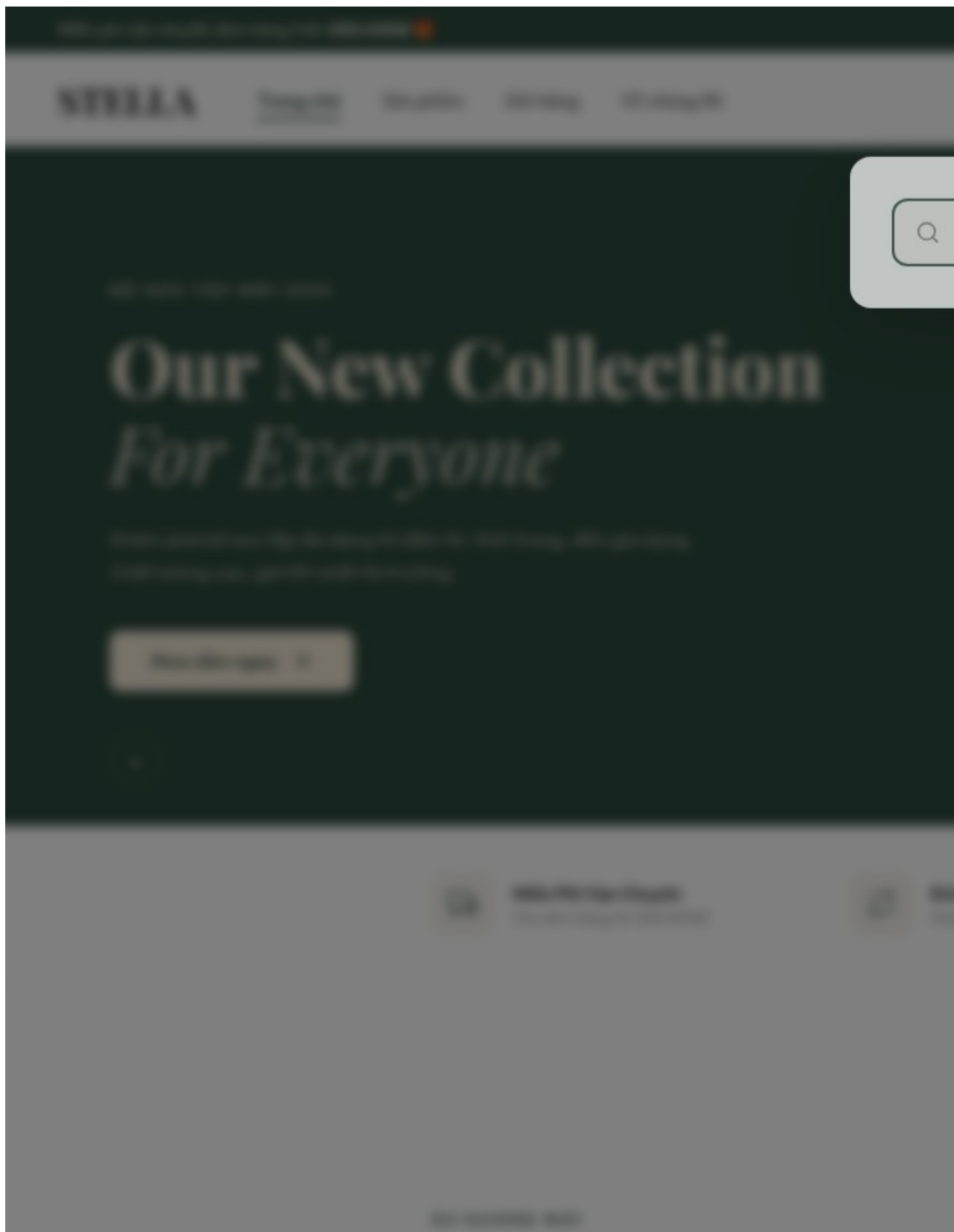
Page object	Trách nhiệm
HomePage	Login, tìm kiếm, thêm sản phẩm, đọc kết quả
CartPage	Xem cart, xóa item, kiểm tra tổng tiền
CheckoutPage	Apply voucher, đặt hàng, kiểm tra warning/error
OrderConfirmationPage	Kiểm tra trang xác nhận đơn hàng
BaseSeleniumTest	Setup Chrome headless, login tự động, screenshot

5.2 Selenium test cases

Test class	Test case	Expected
HomePageTest	Tìm kiếm sản phẩm theo từ khóa	Hiển thị sản phẩm phù hợp
HomePageTest	Tìm kiếm không có kết quả	Hiển thị thông báo trống
CartTest	Thêm nhiều sản phẩm	Badge/cart count cập nhật
CartTest	Xóa sản phẩm khỏi giỏ	Sản phẩm không còn hiển thị
CheckoutFlowTest	Happy path mua hàng với voucher hợp lệ	Trang xác nhận đơn hàng xuất hiện
CheckoutFlowTest	Voucher hợp lệ	Tổng tiền giảm và hiển thị thông báo thành công
CheckoutFlowTest	Voucher không hợp lệ	Hiển thị lỗi
CheckoutFlowTest	Checkout với giỏ trống	Hiển thị cảnh báo

Số test hiện tại: 8. Kết quả trong target/failsafe-reports: 8 pass, 0 failures.

5.3 Minh chứng giao diện





Trang chủ > Sản phẩm

Simple *is More*

DESIGNED TO STAND OUT

LIMITED-EDITIONS STYLES

Hình 2: Thêm nhiều sản phẩm

CHECKOUT

GIỎ HÀNG

/ THANH TOÁN

THÔNG TIN LIÊN LẠC

ĐỊA CHỈ EMAIL

guest.user@example.com

Đã có tài khoản? [Đăng nhập](#)

ĐỊA CHỈ GIAO HÀNG

QUỐC GIA

Việt Nam

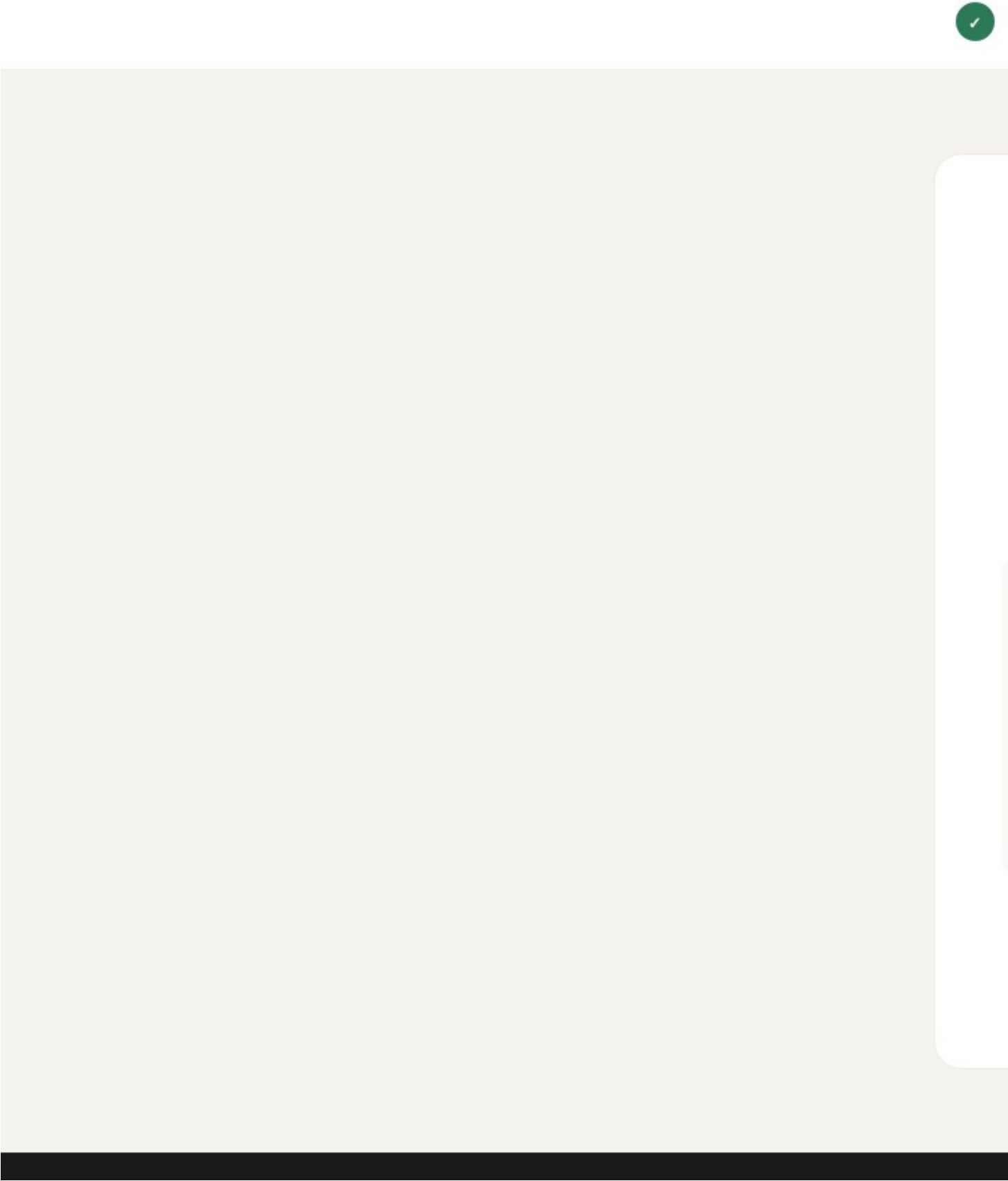
Họ

Nguyễn

ĐỊA CHỈ NHÀ

Hình 3: Áp dụng voucher hợp lệ và hiển thị tổng tiền đã giảm

STELLA



Hình 4: Happy path mua hàng hoàn chỉnh với voucher hợp lệ

6. Công cụ đo chất lượng

6.1 JaCoCo

JaCoCo được cấu hình trong `pom.xml` với các ngưỡng:

Counter	Ngưỡng
Line coverage	$\geq 80\%$
Branch coverage	$\geq 70\%$

Các package `model/config/application class` được loại trừ khỏi báo cáo để tập trung vào `controller, service` và `exception`.

6.2 PIT Mutation Testing

PIT được cấu hình để mutation testing cho `com.ecommerce.service.*` và chạy với unit tests trong `com.ecommerce.unit.*`. Do mutation testing tốn thời gian hơn unit test thông thường, PIT nên chạy riêng khi cần phân tích chất lượng sâu.

Cấu hình chính:

Thiết lập	Giá trị
Target classes	<code>com.ecommerce.service.*</code>
Target tests	<code>com.ecommerce.unit.*</code>
Mutators	STRONGER
Threads	4
Output	HTML, XML

6.3 GitHub Actions

Workflow CI thực hiện:

1. Checkout source.
2. Setup JDK 17.

- 3. Cài Chrome stable.
- 4. Chạy mvn test -B.
- 5. Chạy mvn verify -B.
- 6. Generate JaCoCo report.
- 7. Upload JaCoCo, Surefire và Failsafe reports.

PHẦN IV: BÁO CÁO PHÂN TÍCH TỔNG HỢP (TEST REPORT)

1. Phạm vi thực thi tổng quát

Hạng mục	Nội dung	
Tên dự án	E-Commerce Web Application - Automated Testing Project	
Repository focus	Ứng dụng Spring Boot trong src/	
Thời điểm đối chiếu	Tháng 6/2026	
Công cụ chạy test	Maven Surefire, Maven Failsafe	
Báo cáo nguồn	target/surefire-reports, target/site/jacoco	target/failsafe-reports,
Kết quả tổng quan	84 tests, 84 pass, 0 failures, 0 errors, 0 skipped	

2. Thống kê kết quả theo tầng kiểm thử

Tầng kiểm thử	Test class / Nhóm	Test s	Pas s	Fai l	Erro r	Skipped
Unit	ProductServiceTest	15	15	0	0	0
Unit	CartServiceTest	13	13	0	0	0
Unit	OrderServiceTest	27	27	0	0	0
PBT	PropertyBasedTest	6	6	0	0	0
Integration	HomeControllerTest	4	4	0	0	0

Tầng kiểm thử	Test class / Nhóm	Test s	Pas s	Fai l	Erro r	Skipped
Integration	CartControllerTest	4	4	0	0	0
Integration	CheckoutControllerTest	7	7	0	0	0
E2E	HomePageTest	2	2	0	0	0
E2E	CartTest	2	2	0	0	0
E2E	CheckoutFlowTest	4	4	0	0	0
Tổng	Toàn bộ regression suite	84	84	0	0	0

3. Thống kê kết quả theo phân hệ chức năng

ST T	Phân hệ	Yêu cầu liên quan	Test tiêu biểu	Kết quả
1	Người dùng và bảo mật	REQ-001 -> REQ-004	E2E login flow, SecurityConfig, MockMvc POST	PASS
2	Sản phẩm	REQ-005 -> REQ-010	ProductServiceTest, HomeControllerTest, HomePageTest	PASS
3	Giỏ hàng	REQ-011 -> REQ-015	CartServiceTest, CartControllerTest, CartTest	PASS
4	Đặt hàng và voucher	REQ-016 -> REQ-021	OrderServiceTest, CheckoutControllerTest, CheckoutFlowTest	PASS
5	Trạng thái đơn hàng	REQ-022 -> REQ-027	OrderServiceTest state transition cases	PASS
6	Giao diện sản phẩm	REQ-028 -> REQ-030	Selenium search/no-result, template behavior	PASS

4. Báo cáo coverage

Dữ liệu lấy từ target/site/jacoco/jacoco.csv. Service layer là vùng trọng tâm vì chứa logic nghiệp vụ chính.

Class	Line missed	Line covered	Line coverage	Branch missed	Branch covered	Branch coverage
ProductService	3	27	90.0%	3	19	86.4%
OrderService	1	87	98.9%	2	30	93.8%
CartService	0	35	100.0%	1	13	92.9%
Service layer tổng	4	149	97.4%	6	62	91.2%

Kết luận: service layer vượt ngưỡng exit criteria đã đặt ra: line coverage $\geq 80\%$ và branch coverage $\geq 70\%$.

5. Ma trận truy xuất yêu cầu

Ma trận truy xuất trong docs/traceability-matrix.md cho thấy 30/30 yêu cầu đều có test case hoặc cấu hình cover.

Nhóm yêu cầu	Số yêu cầu	Tình trạng
Quản lý người dùng	4	Covered
Quản lý sản phẩm	6	Covered
Giỏ hàng	5	Covered
Đặt hàng và thanh toán	6	Covered
Trạng thái đơn hàng	6	Covered
Giao diện	3	Covered
Tổng	30	100% covered

6. Defect log

Tại thời điểm đối chiếu báo cáo, bộ automated regression suite không ghi nhận test failure.

ST T	Khu vực	Lỗi ghi nhận từ automation	Mức độ	Trạng thái
1	Unit service layer	Không có lỗi	N/A	PASS
2	Controller layer	Không có lỗi	N/A	PASS
3	E2E Selenium	Không có lỗi	N/A	PASS
4	Coverage threshold	Không vi phạm ngưỡng	N/A	PASS

Các vấn đề còn lại được xếp vào nhóm rủi ro vận hành/thực thi test thay vì defect chức năng:

ST T	Rủi ro	Mô tả	Giải pháp
1	Selenium flaky	Test trình duyệt có thể fail do timing hoặc ChromeDriver	Dùng explicit wait và locator ổn định
2	PIT chạy lâu	Mutation testing tốn nhiều thời gian hơn test thường	Chạy riêng trước demo hoặc trên CI
3	CSRF	Form POST thiếu token có thể lỗi 403	Duy trì th:action, csrf() trong integration test
4	Báo cáo cũ lệch số liệu	Một số tài liệu cũ ghi 82 tests trong khi report hiện tại có 84 tests	Báo cáo này dùng số liệu thực tế từ target/

7. Tổng kết kiểm tra

Chỉ số	Giá trị
Tổng số yêu cầu	30
Yêu cầu đã cover	30
Tỷ lệ requirement coverage	100%

Chỉ số	Giá trị
Unit tests	55
Property-based tests	6
Integration tests	15
E2E Selenium tests	8
Tổng automated tests	84
Tests pass	84
Failures	0
Errors	0
Skipped	0
Service line coverage	97.4%
Service branch coverage	91.2%

8. Đánh giá chất lượng

Bộ kiểm thử hiện tại có cấu trúc phù hợp với testing pyramid: phần lớn test nằm ở tầng unit và property-based để kiểm tra logic nhanh, integration test kiểm tra controller request/response, còn E2E Selenium kiểm tra các luồng người dùng quan trọng.

Các kỹ thuật kiểm thử đã được áp dụng tương đối đầy đủ so với mục tiêu môn học: phân lớp tương đương, phân tích giá trị biên, bảng quyết định, kiểm thử chuyển trạng thái, property-based testing, white-box coverage và mutation testing. Việc kết hợp JaCoCo với PIT giúp đánh giá không chỉ mức độ dòng lệnh được chạy mà còn chất lượng phát hiện lỗi của test suite.

9. Đề xuất cải tiến

ST T	Đề xuất	Lợi ích
1	Bổ sung thêm data-testid cho các phần tử UI quan trọng	Selenium locator ổn định hơn, giảm flaky test
2	Chạy PIT định kỳ trên CI theo lịch hoặc khi release	Đánh giá chất lượng unit test sâu hơn
3	Bổ sung performance smoke test đơn giản cho trang chủ và checkout	Phát hiện sớm suy giảm hiệu năng
4	Tách dữ liệu test E2E rõ hơn theo từng test scenario	Giảm phụ thuộc thứ tự và giảm side effect
5	Tự động publish JaCoCo report artifact trong mỗi lần CI	Dễ theo dõi xu hướng coverage
6	Mở rộng kiểm thử phân quyền admin	Tăng độ tin cậy cho chức năng quản trị sản phẩm

10. Kết luận

Dự án E-commerce WebApp đã xây dựng được bộ kiểm thử tự động đầy đủ cho các chức năng trọng tâm của hệ thống. Kết quả thực thi hiện tại đạt 84/84 automated tests pass, không có failure/error/skipped. Service layer đạt khoảng 97.4% line coverage và 91.2% branch coverage, vượt ngưỡng chất lượng đã đặt ra.

Báo cáo này hoàn thiện lại nội dung theo đúng tình trạng hiện tại của dự án, loại bỏ các chi tiết không khớp như TestNG hoặc Allure/Extent Report, đồng thời bổ sung số liệu thật từ Maven reports, JaCoCo và Selenium screenshots. Nhìn chung, hệ thống đáp ứng tốt mục tiêu kiểm thử tự động của bài tập lớn và có thể tiếp tục mở rộng theo hướng CI/CD, mutation testing định kỳ và kiểm thử hiệu năng.